

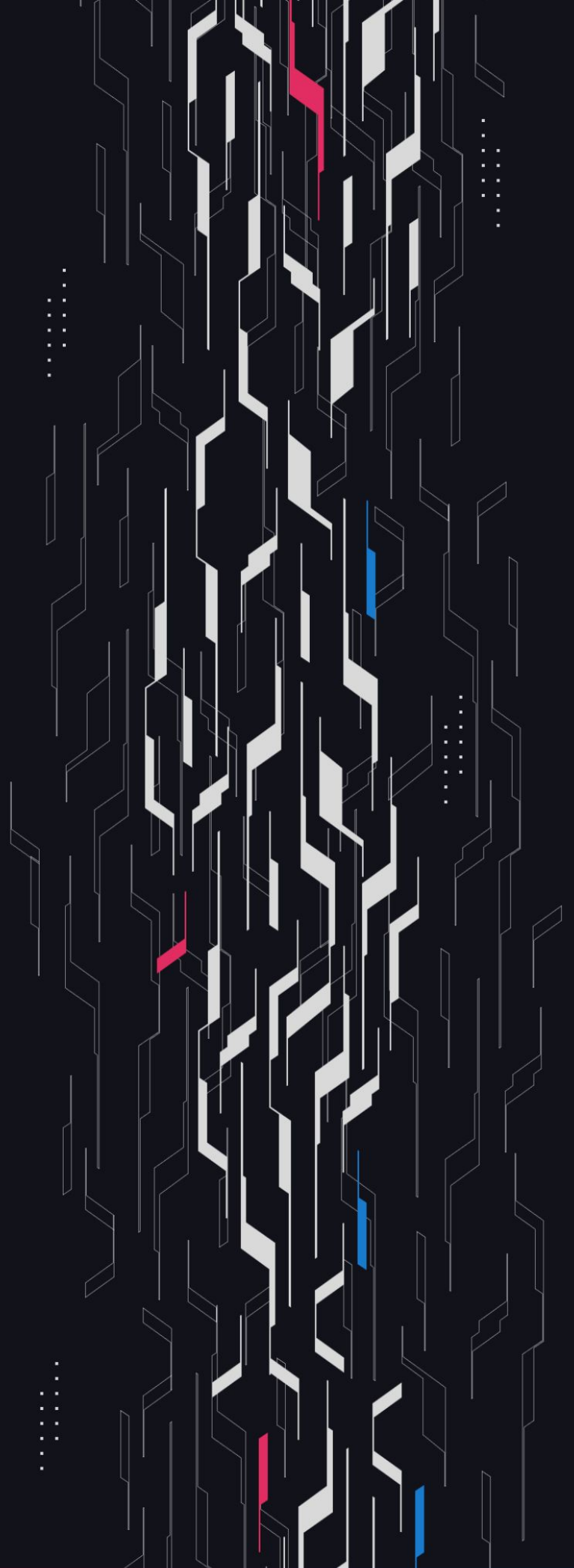
GA GUARDIAN

Reflex

Protocol Review

Security Assessment

May 12th, 2026



Summary

Audit Firm Guardian

Prepared By Minato Namikazi, CrypticDefense

Client Firm Reflex

Final Report Date May 14, 2026

Audit Summary

Reflex engaged Guardian to review the security of their protocol. From March 26 through April 6, 2026, a team of 2 auditors reviewed the source code and recorded the findings in this report.

Confidence Ranking

Given the number issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to address issues thoroughly and consider a targeted follow-up audit depending on code changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Guardian's review scope did not include the Reflex quoter or related off-chain backend infrastructure. Certain acknowledged findings, including [H-01](#) and [H-02](#), are considered mitigated by quoter-side allowlists and operational controls according to the Reflex team. Guardian has not reviewed, audited, or validated the implementation or effectiveness of these quoter-side mitigations, and any related fixes remain outside the scope of this assessment.

A key trust assumption in the Reflex system is that the off-chain quoter correctly restricts allowed pools, tokens, hooks, and route shapes before execution reaches the router. If the quoter is buggy, misconfigured, or compromised, it could emit unsafe routes and break the system's intended guarantees regardless of the on-chain hardening present in the reviewed contracts.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 35

About Guardian 36

Project Overview

Project Summary

Project Name	Reflex
Codebases	https://github.com/reflex-mev/reflex https://github.com/reflex-mev/reflex-full
Commits	Reflex Main Review Commit: 811e29c1ddf536b7e2c2fbcfafe0ac70b9d9a1f9 Reflex Remediation Commit: 20e0abb9e4aa65c607f37fd43cf437c0428e6402 Reflex-Full Main Review Commit: 4858eb85b4dd3dd088519af550fb41fa6335e468 Reflex-Full Remediation Commit: 65a745cb12e920b2b73a544dfe58fffd53dab30b

Audit Summary

Delivery Date	May 12, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	4	0	0	2	0	2
● Medium	2	0	0	2	0	0
● Low	6	0	0	6	0	0
● Info	12	0	0	3	0	9

Audit Scope & Methodology

Scope: Reflex

```
contract, source, total, comment
reflex/core/src/integrations/router/BackrunEnabledSwapProxy.sol, 70, 165, 62
reflex/core/src/integrations/plugin/ReflexAfterSwap.sol, 46, 110, 49
reflex/core/src/integrations/plugin/pancakeswap_infinity/PancakeSwapInfinityHook.sol, 119, 167, 18
reflex/core/src/integrations/plugin/uniswapv4/UniswapV4Hook.sol, 129, 179, 20
source count: {
  total: 621,
  source: 364,
  comment: 149,
  single: 145,
  block: 4,
  mixed: 0,
  empty: 108,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 0.40934065934065933
}
```

Scope: Reflex-Full

```
contract, source, total, comment
reflex-full/core/src/utils/GracefulReentrancyGuard.sol, 23, 63, 31
reflex-full/core/src/base/ConfigurableRevenueDistributor/ConfigurableRevenueDistributor.sol, 126, 217, 54
reflex-full/core/src/base/routers/v2/ExecutionRouter.sol, 378, 716, 254
source count: {
  total: 996,
  source: 527,
  comment: 339,
  single: 201,
  block: 334,
  mixed: 24,
  empty: -229,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 0.643073624
}
```

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	Unvalidated Fallback Enables Profit Theft	Access Control	● High	Acknowledged (mitigated off-chain)
H-02	Missing Access Control On V4/Pancake Callbacks	Access Control	● High	Acknowledged (mitigated off-chain)
H-03	Missing Output Meta Causes Profit Loss	Logical Error	● High	Resolved
H-04	Unfunded Swap Path Leaks Arbitrage Profit	Logical Error	● High	Resolved
M-01	Wrong Profit Token When Initial Hop Is Non-Zero	Logical Error	● Medium	Acknowledged
M-02	tx.origin Used As Profit Recipient In Hooks	Unexpected Behavior	● Medium	Acknowledged
L-01	Native ETH Routes Revert In triggerBackrun	DoS	● Low	Acknowledged
L-02	V4/Pancake Hops Settle Fixed amountIn	DoS	● Low	Acknowledged
L-03	Fee-On-Transfer Token Burns Not Validated	Validation	● Low	Acknowledged
L-04	Silent Uint112 Truncation Of amountIn	Validation	● Low	Acknowledged
I-01	Unvalidated Zero Address Recipient Traps Funds	Validation	● Info	Resolved
I-02	Unused _splitETH Function	Superfluous Code	● Info	Resolved
I-03	withdrawEth Uses Fixed 2300 Gas	Compatibility	● Info	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
I-04	Missing Event In updateDefaultConfig	Events	● Info	Resolved
I-05	Fee Discount Requires Dynamic Fee Configuration	Documentation	● Info	Resolved
I-06	Immutable Owner In Hook Contracts	Configuration	● Info	Resolved
I-07	External Function Has Internal Naming Convention	Best Practices	● Info	Resolved
I-08	Hooks Can Siphon MEV Via Return Delta	Warning	● Info	Acknowledged
I-09	Loan For V2 Without Callback Silently Skipped	Informational	● Info	Resolved

H-01 | Unvalidated Fallback Enables Profit Theft

Category	Severity	Location	Status
Access Control	● High	ExecutionRouter.sol	Acknowledged (mitigated off-chain)

Description

The ExecutionRouter fallback can transfer tokens to msg.sender during the LOAN_CALLBACK_TYPE_ONGOING phase based entirely on caller-controlled calldata, without validating that the caller is the expected pool.

This is dangerous because the router can temporarily hold intermediate route tokens while a swap is in progress. If external execution is reachable during that callback window, an attacker can trigger the fallback, pull those tokens, complete the arbitrage independently, and return enough value for the router’s own flow to finish with zero profit. The loan still repays, but Reflex captures nothing and the attacker keeps the MEV.

Recommendation

Validate msg.sender against the expected callback pool before honoring the LOAN_CALLBACK_TYPE_ONGOING branch. Store the expected callback sender when the swap starts and reject any other caller.

Resolution

Reflex: Acknowledged; mitigated off-chain by the Reflex team.

H-02 | Missing Access Control On V4/Pancake Callbacks

Category	Severity	Location	Status
Access Control	● High	ExecutionRouter.sol	Acknowledged (mitigated off-chain)

Description

unlockCallback and lockAcquired check the router’s callback state, but do not verify that msg.sender is the expected PoolManager or Pancake vault. During the callback window, any contract that can re-enter can reach these handlers.

If a malicious caller reaches the V4/Pancake ongoing-swap path, the router can be induced to settle real tokens to the attacker-controlled contract while treating the fake caller as the swap context. The attacker can then extract the arbitrage profit, return enough assets for the router flow to finish, and leave Reflex with zero captured revenue.

Recommendation

Validate callback callers against the tracked in-lock context or an equivalent expected-caller mapping, and reject any callback that does not come from the legitimate PoolManager or vault.

Resolution

Reflex: Acknowledged; mitigated off-chain by the Reflex team.

H-03 | Missing Output Meta Causes Profit Loss

Category	Severity	Location	Status
Logical Error	● High	ExecutionRouter.sol	Resolved

Description

Standalone V4 and Pancake lock/unlock payloads only carry one token metadata value. The callback then reuses that same metadata as both input and output token info.

When a standalone V4 or Pancake hop outputs native ETH, the router decides whether to wrap based on the wrong token. Instead of wrapping the actual output token, it uses the input token metadata. That can make the route revert during output handling, and in after-swap integrations the revert is swallowed, so the user swap lands but Reflex captures no profit.

Recommendation

Carry tokenMetaOut through every standalone V4 and Pancake lock/unlock payload and callback instead of reusing tokenMetaIn for output handling.

Resolution

Reflex: Resolved.

H-04 | Unfunded Swap Path Leaks Arbitrage Profit

Category	Severity	Location	Status
Logical Error	● High	ExecutionRouter.sol	Resolved

Description

The router explicitly pre-funds the next V2-like hop after a V2/V3 loan, but it does not do the same when a V4 or Pancake hop is followed by a V2-like hop. V4 and Pancake outputs stay on the router, and the next V2-like pair is called without receiving its input tokens first. That makes the next V2-like swap revert. In after-swap integrations the revert is swallowed, so the triggering user swap still lands but Reflex captures nothing. The leftover arbitrage can then be taken by an external searcher.

Recommendation

Centralize the fix in `_swapFlow()` and pre-fund a V2-like hop whenever the previous hop was V4 or Pancake.

Resolution

Reflex: Resolved.

M-01 | Wrong Profit Token When Initial Hop Is Non-zero

Category	Severity	Location	Status
Logical Error	● Medium	ExecutionRouter.sol	Acknowledged

Description

triggerBackrun always extracts the profit token from tokenMeta[0], even though the route can start at any initialHopIndex.

When the initial hop is not zero, the router snapshots the wrong token balance, computes profit against the wrong asset, and distributes nothing. The real profit remains stranded in the contract until an admin withdraws it manually.

Recommendation

Use initialHopIndex when extracting the profit token so profit accounting follows the actual route entryptpoint.

Resolution

Reflex: Acknowledged.

M-02 | tx.origin Used As Profit Recipient In Hooks

Category	Severity	Location	Status
Unexpected Behavior	● Medium	UniswapV4Hook.sol, PancakeSwapInfinityHook.sol	Acknowledged

Description

The hook contracts use tx.origin as the backrun profit recipient and LP-share fallback. That works for simple EOA flows, but breaks for smart wallets, relayers, and account-abstraction setups. In those cases, profits can be misdirected to the bundler, relayer, or submitting signer instead of the intended user or calling contract.

Recommendation

Do not use tx.origin to determine the payout recipient. Pass the intended beneficiary explicitly through the execution flow.

Resolution

Reflex: Acknowledged.

L-01 | Native ETH Routes Revert In triggerBackrun

Category	Severity	Location	Status
DoS	Low	ExecutionRouter.sol	Acknowledged

Description

V4 and Pancake hop execution supports native ETH, but `triggerBackrun` always treats the profit token as an ERC20. If the route starts with native ETH, the router attempts ERC20 balance accounting against `address(0)` and reverts.
This means otherwise valid native-ETH-only routes can fail entirely at profit accounting time.

Recommendation

Add explicit native ETH handling to `triggerBackrun` when the profit token is `address(0)`.

Resolution

Reflex: Acknowledged.

L-02 | V4/Pancake Hops Settle Fixed amountIn

Category	Severity	Location	Status
DoS	● Low	ExecutionRouter.sol	Acknowledged

Description

The V4 and Pancake full-swap paths settle debt using the quoted `amountIn` rather than the actual swap input delta. On partial fills, the real debt can be smaller than the quoted amount. That causes the router to over-settle, leaving an unclaimed credit in the pool manager or vault. The leftover delta can then cause the transaction to revert at unlock finalization.

Recommendation

Settle using the actual `deltaIn` returned by the swap rather than the original quoted input amount.

Resolution

Reflex: Acknowledged.

L-03 | Fee-On-Transfer Token Burns Not Validated

Category	Severity	Location	Status
Validation	● Low	ExecutionRouter.sol: 226-233	Acknowledged

Description

The burn field in tokenMeta is unused everywhere in the router:
All FOT handling is completely dependent on the quoter's pre-adjusted amounts. The router never checks that actual received amounts match expected amounts after burns. Loan repayment transfers a fixed pre-computed amount:
The pool receives less than valid[initialHopIndex] after the burn fee is deducted, which can then fail pool invariant checks, causing a revert. The quoter must predict exactly which transfer path the router takes per hop to compute correct amounts.

Recommendation

Use the burn field to adjust transfer amounts, or implement balance-based accounting rather than relying on pre-computed amounts for FOT tokens.

Resolution

Reflex: Acknowledged.

L-04 | Silent Uint112 Truncation Of amountIn

Category	Severity	Location	Status
Validation	● Low	ReflexAfterSwap.sol: 162	Acknowledged

Description

In ReflexAfterSwap::_reflexAfterSwap, amountIn is cast from uint256 to uint112 without validation: If amountIn exceeds type(uint112).max (~5.19e33), the value silently truncates, passing an incorrect amount to the quoter. The quoter computes a route based on the wrong amount, resulting in either no profit found or a suboptimal arbitrage. Due to the try-catch, the MEV opportunity is silently missed. While overflow is unlikely for standard 18-decimal tokens, it becomes realistic, for example, for non-standard high-decimal tokens.

Recommendation

Add an overflow check before the cast:

Resolution

Reflex: Acknowledged.

I-01 | Unvalidated Zero Address Recipient Traps Funds

Category	Severity	Location	Status
Validation	● Info	ConfigurableRevenueDistributor.sol: 137	Resolved

Description

In triggerBackrun, the recipient parameter is passed directly to _splitERC20 as variedRecipient without any zero-address check:
When variedRecipient is address(0), the varied share distribution block in _splitERC20 is skipped entirely:
With the default config allocating 80% to main recipients and 20% to the varied recipient, passing address(0) silently leaves 20% of profit plus undistributed in the contract. The admin can recover via withdrawToken, but there is no indication to the caller that funds were partially retained.

Recommendation

Validate recipient is non-zero in triggerBackrun:

Resolution

Reflex: Resolved.

I-02 | Unused _splitETH Function

Category	Severity	Location	Status
Superfluous Code	● Info	ConfigurableRevenueDistributor.sol: 160	Resolved

Description

ConfigurableRevenueDistributor::_splitETH is implemented but never called anywhere in the protocol. ReflexRouter::triggerBackrun uses _splitERC20 for profit distribution, and no other code path invokes _splitETH. Note that if it were ever integrated for native ETH profit distribution, it also introduces reentrancy risk via recipient.call{value: share}("").

Recommendation

Remove _splitETH if native ETH distribution is not intended. If it is planned for future use, ensure to add reentrancy protection.

Resolution

Reflex: Resolved.

I-03 | withdrawEth Uses Fixed 2300 Gas

Category	Severity	Location	Status
Compatibility	● Info	ExecutionRouter.sol: 705	Resolved

Description

withdrawEth uses .transfer() which forwards only 2300 gas to the recipient:
This may fail if _to is a contract that requires more than 2300 gas to receive ETH.

Recommendation

Use a low-level call instead:

Resolution

Reflex: Resolved.

I-04 | Missing Event In updateDefaultConfig

Category	Severity	Location	Status
Events	● Info	ConfigurableRevenueDistributor.sol: 104	Resolved

Description

ConfigurableRevenueDistributor::updateShares emits SharesUpdated after modifying a configuration, but updateDefaultConfig does not:
Off-chain indexers and systems tracking SharesUpdated events will not detect changes to it from the default configuration.

Recommendation

Emit SharesUpdated in updateDefaultConfig.

Resolution

Reflex: Resolved.

I-05 | Fee Discount Requires Dynamic Fee Configuration

Category	Severity	Location	Status
Documentation	● Info	UniswapV4Hook.sol: 118	Resolved

Description

In both `UniswapV4Hook` and `PancakeSwapInfinityHook`, `beforeSwap` returns `LPFeeLibrary.OVERRIDE_FEE_FLAG` to grant the router zero fees on backrun swaps: This override is only in effect if the pool was initialized with dynamic fees enabled in its pool key. If the pool uses static fees, the flag is silently ignored and the router pays the full LP fee on every backrun, reducing captured MEV profit for users.

Recommendation

Document that pools integrating Reflex hooks must be initialized with dynamic fees enabled for the fee discount to function. Consider adding a validation check during pool initialization to ensure compatibility.

Resolution

Reflex: Resolved.

I-06 | Immutable Owner In Hook Contracts

Category	Severity	Location	Status
Configuration	● Info	UniswapV4Hook.sol: 23	Resolved

Description

In both UniswapV4Hook and PancakeSwapInfinityHook, owner is declared immutable. If the owner key needs to be rotated for security reasons or operational changes, the existing hook's admin functions (router updates, config changes, fee discounts) cannot be transferred to a the new owner.

Recommendation

Implement a transferOwnership function with a two-step transfer pattern to allow safe owner rotation.

Resolution

Reflex: Resolved.

I-07 | External Function Has Internal Naming Convention

Category	Severity	Location	Status
Best Practices	● Info	PancakeSwapInfinityHook.sol: 176	Resolved

Description

`_donateToPool` in both `UniswapV4Hook` and `PancakeSwapInfinityHook` is declared external but uses an underscore prefix, which by Solidity convention indicates internal or private visibility: The function is external to enable try-catch on self-calls, however, the underscore prefix is misleading and may cause confusion during future development.

Recommendation

Rename to `donateToPool` to reflect its external visibility.

Resolution

Reflex: Resolved.

I-08 | Hooks Can Siphon MEV Via Return Delta

Category	Severity	Location	Status
Warning	● Info	ExecutionRouter.sol: 379	Acknowledged

Description

The router executes arbitrage routes through any V4 or PancakeSwap pool the quoter selects, however, V4 and PancakeSwap hooks can modify swap return deltas through `beforeSwap/afterSwap` (including adjusting deltas, taking fees, or making external calls). A hook on an intermediate pool in a route could reduce the swap output, effectively siphoning a portion of the expected arbitrage profit. The router does not validate actual swap output against expected amounts, so reduced output from a hook goes undetected as long as the remaining amount is sufficient to complete the route and repay the loan.

Recommendation

Document the trust assumptions around routing through UniswapV4 and PancakeSwap pools, while ensuring the quoter only routes through pools with verified and trusted hooks.

Resolution

Reflex: Acknowledged.

I-09 | Loan For V2 Without Callback Silently Skipped

Category	Severity	Location	Status
Informational	● Info	ExecutionRouter.sol: 292	Resolved

Description

_triggerSwapRoute has no branch handling isUniswapV2WithoutCallback as the initial hop. V2 without callback pools cannot execute the initial hop since they lack the data parameter needed for flash swaps. If the quoter selects one, the function silently does nothing (no swap executes, profit = 0, and gas is wasted).

Recommendation

Ensure the quoter does not return isUniswapV2WithoutCallback as the initial hop, or add an explicit check/revert to fail gracefully rather than silently revert if this occurs.

Resolution

Reflex: Resolved.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Backrun Failure Return Value Is Ambiguous	Unexpected Behavior	 Low	Acknowledged
I-01	Admin Changes Can Silently Skip MEV Capture	Trust Assumptions	 Info	Acknowledged
I-02	Native ETH Donate Fails In Hook Contracts	Informational	 Info	Acknowledged
I-03	Redundant Add Operation In _bytesToAddress	Superfluous Code	 Info	Resolved

L-01 | Backrun Failure Return Value Is Ambiguous

Category	Severity	Location	Status
Unexpected Behavior	● Low	BackrunEnabledSwapProxy.sol: 163-167	Acknowledged

Description

In `BackrunEnabledSwapProxy::swapWithBackrun`, the dev comments state that failed backruns are identified by zero return values:
However, `triggerBackrun` returns `(0, address(0))` on a successful execution when no arbitrage opportunity exists:
This makes a successful, although unprofitable, backrun execution identical to a caught revert/failure. This will affect callers attempting to distinguish between a route with no arbitrage available and a broken route that consistently fails, preventing appropriate retry and error handling.

Recommendation

Consider handling the case where a successful `triggerBackrun` call returns `(0, address(0))` as success rather than unexpected failure.

Resolution

Reflex: Acknowledged.

I-01 | Admin Changes Can Silently Skip MEV Capture

Category	Severity	Location	Status
Trust Assumptions	● Info	ReflexAfterSwap.sol: 100	Acknowledged

Description

If the admin calls `setReflexRouter` or modifies fee discounts via `setGlobalFeeDiscount/setPoolFeeDiscount` while a user's swap transaction is pending, the backrun may silently fail. The previous router loses its fee discount, causing the backrun to pay full LP fees, which can eliminate the arbitrage profit. The try-catch in `_reflexAfterSwap` skips the failure, and the user's swap completes normally but without receiving any MEV redistribution.

Recommendation

Consider documenting that admin operations on hook contracts should be performed during low-activity periods to minimize impact on pending user swap

Resolution

Reflex: Acknowledged.

I-02 | Native ETH Donate Fails In Hook Contracts

Category	Severity	Location	Status
Informational	● Info	UniswapV4Hook.sol: 202-211	Acknowledged

Description

In `donateToPool` within both `UniswapV4Hook` and `PancakeSwapInfinityHook`, the `WETH/ETH` branch only matches when `currency` is native `ETH` and `profitToken` is `WETH`:
If both `currency` and `profitToken` are `address(0)` (native `ETH`), the first branch is skipped since `profitToken != weth`, and the `else` branch calls `IERC20(address(0)).safeTransfer` which reverts. Currently, sponsors have confirmed that profit token is not expected to be native `ETH`. However, if this changes in the future, the current hook contracts are incompatible and will cause unexpected reverts.

Recommendation

Consider adding handling for the case where both `profitToken` and `currency` are native `ETH`, or document that if `ETH` is added as a profit token in the future, the `UniswapV4` and `PancakeSwap` hook contracts are incompatible.

Resolution

Reflex: Acknowledged.

I-03 | Redundant Add Operation In _bytesToAddress

Category	Severity	Location	Status
Superfluous Code	● Info	ExecutionRouter.sol: 794	Resolved

Description

_bytesToAddress contains a redundant add operation in its assembly block. add(add(d, 20), 0) is functionally identical to add(d, 20), wasting an opcode.

Recommendation

Simplify to addr := mload(add(d, 20)).

Resolution

Reflex: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>